

A
Project Report
On

“IoT-Enabled ADAS Using CAN Protocol”

Submitted in the partial fulfillment of the requirements
for the PG Diploma in

**EMBEDDED SYSTEMS & DESIGN
(PG - DESD)**

by

Ashitosh Vishvas Wani	250844230012
Bhavesb Pravin Mahajan	250844230046
Pratham Rajkumar Kale	250244230068
Vishwesh Avinash Patil	250244230058



Sunbeam Institute of Information Technology,
Pune & Karad.
Year: Aug - 2025

**Sunbeam Institute of Information Technology,
Pune & Karad.**



CERTIFICATE

This is to certify,

Ashitosh Vishvas Wani

250844230012

Bhavesb Pravin Mahajan

250844230046

Pratham Rajkumar Kale

250244230068

Vishwesh Avinash Patil

250244230058

have satisfactorily completed Project and presented a report on topic titled “IoT-Enabled ADAS System Using CAN Protocol” at Sunbeam Institute of Information Technology in partial fulfilment of requirement of PG Diploma in Embedded Systems & Design (PG-DESD) academic year 2025.

Date: 31/01/2026

ACKNOWLEDGEMENT

This project, “**IoT-Enabled ADAS System Using CAN Protocol**,” was a great learning experience for us, and we are submitting this work to Sunbeam Institute of Information Technology. We are very grateful to **Ms. Utkarsha Mam** for her valuable guidance throughout the project. Her continuous support and encouragement helped us overcome various challenges and technical difficulties during the course of the project work.

We would also like to express our sincere thanks to **Mr. Devendra Dhande (Course Coordinator, PG-DESD)** for providing all the necessary support and coordination, including access to required hardware, internet facilities, and extended lab hours, which were essential for the successful completion of this project and throughout the course duration at Sunbeam Institute of Information Technology, Pune.

INDEX

Sr no.	Contents	Page no.
1	Introduction	1
1.1	Project Scope	1
1.2	CAN Protocol	1
2	Hardware Requirements	4
3	Software Requirements	13
4	Design and Implementation	15
5	Advantages	18
6	Disadvantages	19
7	Future Scope	20
8	Conclusion	21
9	Reference	22

LIST OF FIGURES

Sr No.	Figures	Page No.
1.1	CAN Frame Format	2
2.1	STM32F407 Discovery Board	5
2.2	MCP2551 CAN.....	6
2.3	L298N Motor Driver	7
2.4	HC-SR04 Ultrasonic Sensor	9
2.5	Timing Diagram.....	9
2.6	DC Motor.....	11
2.7	IR Speed Sensor.....	12
2.8	Fule Level Sensor	14
2.9	UART	12
3.1	STMCubeIDE	14
4.1	Block Diagram.....	15
4.2	Circuit Diagram	17
8.1	ADAS System Implemented on STM32.....	21

LIST OF TABLES

Sr No.	Table	Page No.
2.1	Motor States.....	7
2.2	Electric Parameters of HCSR04.....	8

ABSTRACT

This project focuses on the design and implementation of an Advanced Driver Assistance System (ADAS) using the Controller Area Network (CAN) protocol for reliable in-vehicle communication. The system is developed using two STM32F407 microcontrollers, where one acts as a Sensor ECU and the other as a Control ECU. The Sensor ECU measures obstacle distance using an ultrasonic sensor and vehicle speed using an IR sensor, and transmits this data over the CAN bus. The Control ECU receives the data, processes safety logic, and controls a DC motor through an L298N motor driver to simulate automatic braking when an obstacle is detected within a predefined threshold. CAN protocol ensures fast, deterministic, and noise-immune communication between the ECUs, which is essential for automotive safety applications. UART is used for real-time monitoring and debugging of received data. This project demonstrates a simple and effective CAN-based ADAS feature similar to automatic emergency braking systems used in modern vehicles, and it can be further extended by integrating additional sensors and intelligent control algorithms for enhanced safety and performance.

1. INTRODUCTION

About Project:

This project aims to improve vehicle safety by developing an Advanced Driver Assistance System based on automatic obstacle detection and braking using the Controller Area Network protocol. The system uses an ultrasonic sensor to detect obstacles in front of the vehicle. The sensed data is processed by an STM32F407 microcontroller, which transmits distance and speed information over the CAN bus to a control unit. When a potential collision is detected, the control unit activates the braking action by stopping the motor through a motor driver circuit. The CAN protocol ensures fast, reliable, and noise-immune communication between system components, which is essential for automotive applications. A buzzer is also used to provide an audible warning during hazardous conditions. By combining sensor-based detection with CAN-based communication, the system helps reduce collision risk and enhances overall vehicle safety.

1.1 Project scope:

The scope of this project is to design and implement a CAN-based Advanced Driver Assistance System that focuses on obstacle detection and automatic braking to enhance vehicle safety. The system uses an ultrasonic sensor to continuously monitor the distance between the vehicle and obstacles and processes this data using an STM32F407 microcontroller. Based on the received sensor data, braking action is simulated by controlling a DC motor through a motor driver when the obstacle distance falls below a predefined threshold.

The project also includes vehicle parameter monitoring such as speed measurement using an IR sensor and system status indication through alerts. CAN protocol is used to ensure reliable, real-time communication between multiple electronic control units, reflecting a practical automotive network architecture. The scope of this project is limited to demonstrating core ADAS functionality and does not include advanced sensing technologies such as radar or camera-based vision systems, which can be considered for future enhancement.

1.2 CAN Protocol:

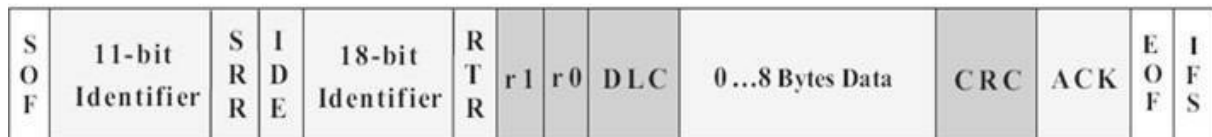
The CAN bus was developed by BOSCH as a multi-master, message broadcast system that specifies a maximum signaling rate of 1 megabit per second (bps). Unlike a traditional network such as USB or Ethernet, CAN does not send large blocks of data point-to-point from node A to node B under the supervision of a central bus master. In a CAN network, many short messages like temperature or RPM are broadcast to the entire network, which provides for data consistency in every node of the system. CAN is an International Standardization Organization (ISO)-defined serial communications bus originally developed for the automotive industry to replace the complex wiring harness with a two-wire bus.

The specification calls for high immunity to electrical interference and the ability to self-diagnose and repair data errors.

CAN Frame Format:



(a)



(b)

Fig 1.1 CAN Frame Format.

The following figure shows the CAN protocols' frame format.

- SOF: The single dominant start of frame (SOF) bit marks the start of a message and is used to synchronise the nodes on a bus after being idle.
- Identifier: The standard CAN 11-bit identifier establishes the priority of the message. The lower the binary value, the higher its priority.
- RTR: The single remote transmission request (RTR) bit is dominant when information is required from another node. All nodes receive the request, but the identifier determines the specified node. The responding data is also received by all nodes and used by any node interested. In this way, all data being used in a system is uniform.
- IDE: A dominant single identifier extension (IDE) bit means that a standard CAN identifier with no extension is being transmitted.
- ro: Reserved bit (for possible use by future standard amendment).
- DLC: The 4-bit data length code (DLC) contains the number of bytes of data being transmitted.
- Data: up to 64 bits of application data may be transmitted.
- CRC: The 16-bit (15 bits plus delimiter) cyclic redundancy check (CRC) contains the checksum (number of bits transmitted) of the preceding application data for error detection.
- ACK: Every node receiving an accurate message overwrites this recessive bit in the original message with a dominant bit, indicating an error-free message has been sent. Should a receiving node detect an error and leave this bit recessive, it discards the message, and the sending node repeats the message after re-arbitration. In this node acknowledges (ACK) the integrity of its data. ACK is 2 bits; one is the acknowledgement bit and the second is a delimiter. way, each
- EOF: This end-of-frame (EOF), 7-bit field marks the end of a CAN frame (message) and disables bit-stuffing, indicating a stuffing error when dominant. When 5 bits of the same logic level occur in succession during normal operation, a bit of the opposite logic level is stuffed into the data.
- IFS: This 7-bit inter-frame space (IFS) contains the time required by the controller to move a correctly received frame to its proper position in a message buffer area.

The Extended CAN Frame format consists of these additional bits:

- SRR: The substitute remote request (SRR) bit replaces the RTR bit in the standard message location as a placeholder in the extended format.
- IDE-A recessive bit in the identifier extension (IDE) indicates that more identifier bits follow. The 18-bit extension follows IDE.
- r1: Following the RTR and r0 bits, an additional reserve bit has been included ahead of the DLC bit.

System Requirements

Hardware requirements are as follows:

1. 2x STM32F407 Discovery Board.
2. 2x CAN transceivers (MCP2551)
3. 1x Ultrasonic Sensor (HCSR04)
4. 1x Motor Drive module (L298N)
5. 1x DC motor
6. 1x Speed Sensor
7. 1x UART
8. 2x 120 Ω Resistor
9. 1x ESP32

Software requirements are as follows:

1. The software we have used for implementing the project is STM32Cube IDE 1.18.0.
2. Arduino IDE (for ESP32 / ThingSpeak integration)
3. ThingSpeak cloud platform

2. HARDWARE REQUIREMENTS

1. STM32F407 Discovery Board

The STM32F407G-DISC1 Discovery Kit is designed to help both beginners and experienced users explore the features of the STM32F407/417 line and develop applications. It's based on the STM32F407VGT6 microcontroller and includes an embedded ST-LINK/V2 debug tool, MEMS sensors, an audio DAC, LEDs, push buttons, and a USB OTG micro-AB connector.

Key Features and Specifications:

- Microcontroller: STM32F407VGT6 featuring a 32-bit ARM Cortex-M4F core, 1 MB Flash, and 192 KB RAM in an LQFP100 package. The core runs at up to 168 MHz and includes a floating-point unit (FPU) and Digital Signal Processing (DSP) instructions for high-performance embedded applications.
- On-board ST-LINK/V2: With a selection mode switch, it can be used as a standalone ST-LINK/V2 with an SWD connector for programming and debugging. Newer boards use ST-LINK/V2-A.
- Power Supply: Can be powered through a USB bus or an external 5 V supply voltage. It also provides an external application power supply at 3 V and 5 V.
- MEMS: Includes a LIS302DL or LIS3DSH ST MEMS 3-axis accelerometer and an MP45DT02 ST MEMS audio sensor, which is an omnidirectional digital microphone.
- LEDs: Eight LEDs are present: LD1 (red/green) for USB communication, LD2 (red) for 3.3 V power on, four user LEDs (orange, green, red, and blue)
- Push Buttons: It has two push buttons for user input and reset.
- USB OTG FS: Comes with a micro-AB connector.
- Debug and Programming: The ST-LINK circuitry allows you to flash code onto the microcontroller and debug it using host software like Keil or STM Cube IDE.

The board comes in two versions, a newer one labeled "DISC1" and an older one without this designation. The board identification number MB997D is for the newer board, while MB997C is for older versions.

The STM32F4 Discovery board allows users to develop and design applications, offering modules for communication and interface design without relying on third-party devices. It incorporates modern system modules and peripherals like DAC, ADC, and UART.

It supports a wide choice of Integrated Development Environments (IDEs) including IAR Embedded Workbench®, MDK-ARM, and STM32CubeIDE

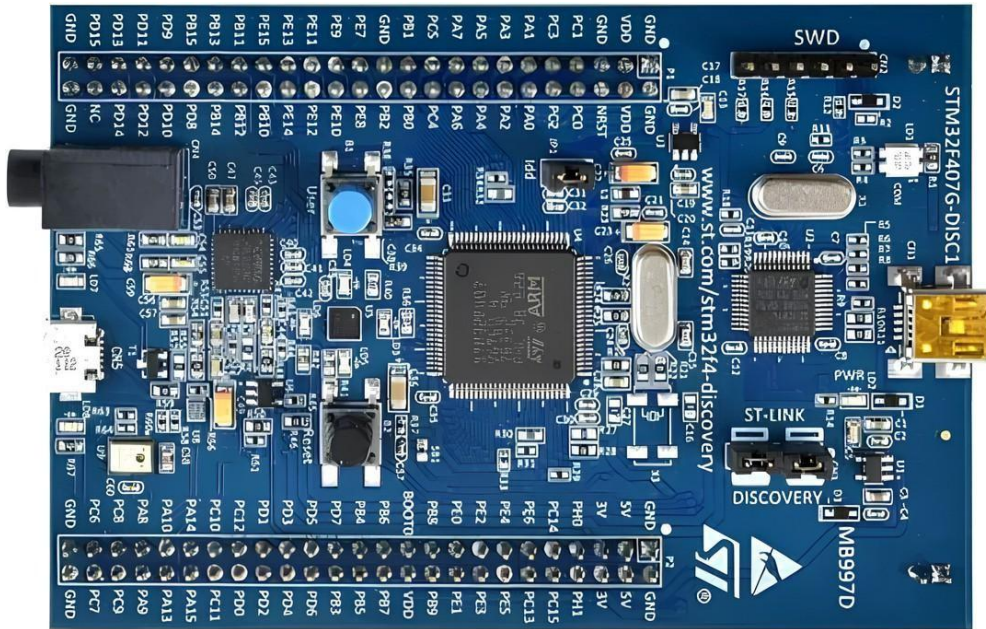


Fig 2.1 STM32F407 Discovery Board

2. MCP2551 CAN Transceivers:

The MCP2551 is a high-speed CAN, fault-tolerant device that serves as the interface between a CAN protocol controller and the physical bus. The MCP2551 device provides differential transmit and receive capability for the CAN protocol controller, and is fully compatible with the ISO-11898 standard, including 24V requirements. It will operate at speeds of up to 1 Mb/s. Typically, each node in a CAN system must have a device to convert the digital signals generated by a CAN controller to signals suitable for transmission over the bus cabling (differential output). It also provides a buffer between the CAN controller and the high-voltage spikes that can be generated on the CAN bus by outside sources.

Some of its features are:

- Supports 1 Mb/s operation
- Implements ISO-11898 standard physical layer requirements
- Suitable for 12V and 24V systems
- Detection of ground fault (permanent Dominant) on TXD input
- Power-on Reset and voltage brown-out protection
- An unpowered node or brown-out event will not disturb the CAN bus
- Low current standby operation
- Protection against damage due to short-circuit conditions (positive or negative battery voltage)
- Up to 112 nodes can be connected
- High-noise immunity due to differential bus implementation
- Temperature ranges: - Industrial (I): -40°C to +85°C - Extended (E): -40°C to +125°C



Fig 2.2 MCP255

3. L298N Motor Drive Module.

The L298N Motor Driver Module is a high-power module designed for driving DC and Stepper Motors, containing an L298 motor driver IC and a 78M05 5V regulator. It can control up to four DC motors or two DC motors with directional and speed control.

Pinout Configuration:

- IN1 & IN2 : Motor A input pins for controlling spinning direction.
- IN3 & IN4 : Motor B input pins for controlling spinning direction.
- ENA : Enables PWM signal for Motor A.
- ENB : Enables PWM signal for Motor B.
- OUT1 & OUT2 : Output pins of Motor A.
- OUT3 & OUT4 : Output pins of Motor B.
- 12V : 12V input from DC power source.
- 5V : Supplies power for the switching logic circuitry inside L298N IC.
- GND : Ground pin.

Features & Specifications:

- Driver Model: L298N 2A
- Driver Chip: Double H Bridge L298N
- Motor Supply Voltage (Maximum): 46V
- Motor Supply Current (Maximum): 2A
- Logic Voltage: 5V
- Driver Voltage: 5-35V
- Driver Current: 2A
- Logical Current: 0-36mA
- Maximum Power (W): 25W
- Current Sense for each motor
- Heatsink
- Power-On LED indicator

IoT-Enabled ADAS System Using CAN Protocol

The control mode and state of motor:

ENA	IN1	IN2	STATE
1	0	0	Brake
1	0	1	Rotate Clockwise
1	1	0	Rotate Anti-Clockwise
1	1	1	Brake

Table 2.1. Motor States.

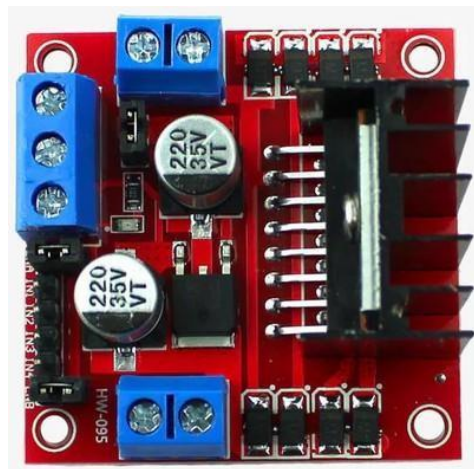


Fig. 2.3 L298N

4. Ultrasonic Sensor (HC-SR04)

Ultrasonic ranging module HC - SR04 provides 2cm - 400cm non-contact measurement function, the ranging accuracy can reach to 3mm. The modules include ultrasonic transmitters, receiver and control circuit. The basic principle of work:

- 1) Using IO trigger for at least 10us high level signal,
- 2) The Module automatically sends eight 40 kHz and detects whether there is a pulse signal back.
- 3) IF the signal back, through high level, time of high output IO duration is the time from sending ultrasonic to returning.

Test distance = (high level time velocity of sound (340M/S)/2

Wire connecting direct as following:

- 5V Supply
- Trigger Pulse Input
- Echo Pulse Output
- 0V Ground

Working Voltage	DC 5 V
Working Current	15 mA
Working Frequency	40 KHz
Max Range	4m
Min Range	2cm
Measuring Angle	15 degrees
Trigger Input Signal	10uS TTL pulse
Echo Output Signal	Input TTL lever signal and the range in Proportion
Dimension	45*20*15mm

Table 2.2. Electric parameters of HC-SR04



Fig 2.4. HCSR04

Timing diagram:

The timing diagram is shown below. You only need to supply a short 10 μ s pulse to the trigger input to start the ranging, and then the module will send out an 8-cycle burst of ultrasound at 40 kHz and raise its echo. The Echo is a distance object that is pulse width and the range in proportion. You can calculate the range through the time interval between sending the trigger signal and receiving the echo signal. Formula: $\mu s / 58 = \text{centimeters}$ or $\mu s / 148 = \text{inches}$; or: the range = high-level time * velocity (340 m/s) / 2; we suggest using over a 60 ms measurement cycle in order to prevent the trigger signal from reaching the echo signal.

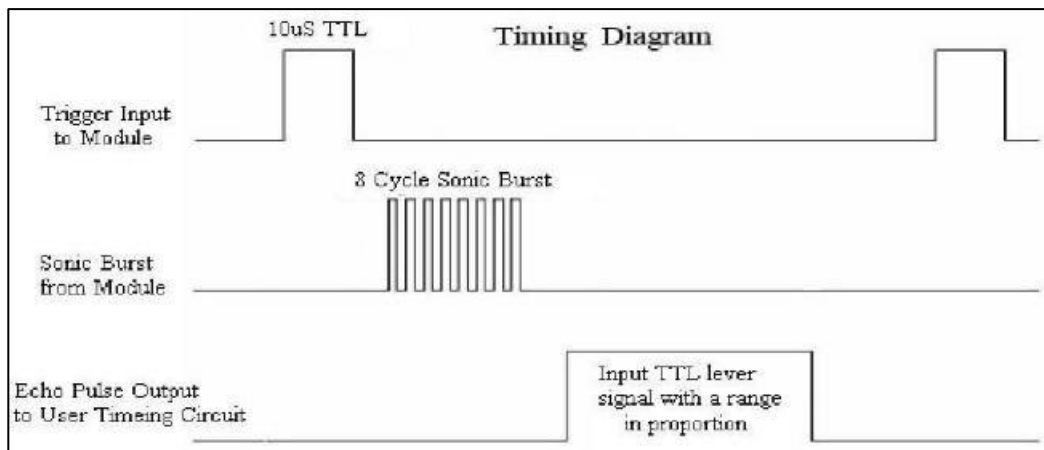


Fig 2.5. Timing Diagram of HCSR-04

5. DC Motor.

Almost every mechanical movement that we see around us is accomplished by an electric motor. Electric machines are a means of converting energy. Motors take electrical energy and produce mechanical energy. Electric motors are used to power hundreds of devices we use in everyday life. Motors come in various sizes. Huge motors that can take loads of 1000s of horsepower are typically used in the industry. Some examples of large motor applications include elevators, electric trains, hoists, and heavy metal rolling mills. Examples of small motor applications include motors used in automobiles, robots, hand power tools, and food blenders. Micro-machines are electric machines with parts the size of red blood cells and find many applications in medicine. Electric motors are broadly classified into two different categories: DC (Direct Current) and AC (Alternating Current). Within these categories are numerous types, each offering unique abilities that suit them well for specific applications. In most cases, regardless of type, electric motors consist of a stator (stationary field) and a rotor (the rotating field or armature) and operate through the interaction of magnetic flux and electric current to produce rotational speed and torque. DC motors are distinguished by their ability to operate from direct current. There are different kinds of D.C. motors, but they all work on the same principles. In this chapter, we will study their basic principle of operation and their characteristics. It's important to understand motor characteristics so we can choose the right one for our application requirements. The learning objectives for this chapter are listed below.

DC Motor Basic Principles

- Energy Conversion

If electrical energy is supplied to a conductor lying perpendicular to a magnetic field, the interaction of current flowing in the conductor and the magnetic field will produce mechanical force (and therefore, mechanical energy).

- Value of Mechanical Force

There are two conditions that are necessary to produce a force on the conductor. The conductor must be carrying current and must be within a magnetic field. When these two conditions exist, a force will be applied to the conductor, which will attempt to move the conductor in a direction perpendicular to the magnetic

field. This is the basic theory by which all DC motors operate. The force exerted upon the conductor can be expressed as follows.

$$F = B * i * l \text{ Newton}$$

where B is the density of the magnetic field, l is the length of the conductor, and i is the value of the current flowing in the conductor. The direction of motion can be found using Fleming's Left Hand Rule.

The first finger points in the direction of the magnetic field (first - field), which goes from the North Pole to the South Pole. The second finger points in the direction of the current in the wire (second - current). The thumb then points in the direction the wire is thrust or pushed while in the magnetic field (thumb: torque or thrust).



Fig 2.6. DC Motor

6. UART.

UART stands for Universal Asynchronous Receiver Transmitter. It is a serial communication interface used to exchange data between digital devices without using a shared clock signal. Communication is asynchronous, meaning data is sent in frames consisting of a start bit, data bits, optional parity bit, and stop bits.

UART uses two main signal lines: TX (transmit) and RX (receive), along with a common ground. Typical UART configurations include 8 data bits, no parity, and 1 stop bit, with commonly used baud rates such as 9600 or 115200 bps. UART is simple, reliable, and widely used in embedded systems for device communication, configuration, and debugging purposes.



Fig 2.7. UART

7. IR Speed Sensor.

The IR speed sensor is used to measure the rotational speed (RPM) of the motor or wheel by detecting interruptions caused by a rotating object. It consists of an infrared transmitter and receiver pair along with a signal conditioning circuit, typically based on the LM393 comparator. When an object passes between the IR transmitter and receiver, it generates a digital pulse.

Key features and specifications:

Sensor type: Infrared reflective / obstacle-based speed sensor

Operating voltage: 3.3 V to 5 V DC

Output type: Digital pulse output

Comparator IC: LM393

Current consumption: Low power consumption

Detection method: Interrupt-based pulse detection

Response time: Fast response suitable for RPM measurement

Pin configuration: VCC, GND, Digital Output

Working principle:

The IR sensor emits infrared light continuously. When a rotating object (such as a wheel or encoder disc) passes in front of the sensor, the reflected IR light is detected by the receiver, generating a pulse. These pulses are counted by the microcontroller using an interrupt pin. By counting the number of pulses in a fixed time interval, the rotational speed (RPM) is calculated using a predefined pulses-per-revolution value.

Applications:

The IR speed sensor is widely used in speed measurement of motors, wheels, and rotating machinery in automotive systems, robotics, industrial automation, and embedded control applications due to its simplicity, reliability, and ease of interfacing with microcontrollers.

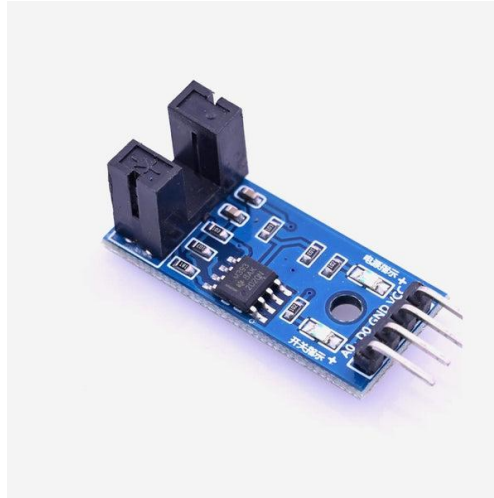


Fig 2.6. LM393 Speed Sensor

8. Water/ Fuel Level Sensor.

The fuel level sensor used in this project is a conductivity-based liquid level sensor, adapted for demonstration purposes. It detects the presence and approximate level of liquid using a separate sensing probe with exposed conductive tracks. As the liquid level increases, the conductivity between the tracks changes, allowing the sensor to indicate different fuel level conditions.

Key features and specifications:

- Sensor type: Conductivity-based liquid level sensor (probe type)
- Operating voltage: 3.3 V to 5 V DC
- Output: Analog signal proportional to liquid level
- Detection method: Conductivity change across exposed sensor tracks
- Current consumption: Low power operation
- Pin configuration: 3 pins — VCC, GND, Signal
- Operating principle: Liquid presence increases conductivity between probe tracks, changing output voltage
- Interface compatibility: Compatible with microcontrollers such as STM32, Arduino, and ESP32 through ADC inputs

- Application usage: Used for conceptual fuel level monitoring in prototype and educational projects.
- Limitation: Suitable only for demonstration with conductive liquids, not for real automotive fuel systems

This sensor is used in the project to demonstrate the concept of fuel level monitoring within the IoT-enabled ADAS system.



Fig 2.7 Fuel Level Sensor

9. ESP32

STM32CubeIDE is a free, all-in-one integrated development environment (IDE) ESP32 is a low-cost, high-performance microcontroller developed by Espressif Systems, designed specifically for IoT and wireless embedded applications. It integrates a powerful processor with built-in Wi-Fi and Bluetooth, making it suitable for cloud-connected systems.

Key features and specifications:

- Processor: Dual-core 32-bit Xtensa LX6 microprocessor
- Clock frequency: Up to 240 MHz
- Wireless connectivity: Integrated Wi-Fi (802.11 b/g/n) and Bluetooth (Classic and BLE)
- Operating voltage: 3.0 V to 3.6 V (typically 3.3 V)
- Flash memory: External SPI flash (commonly 4 MB)
- SRAM: Approximately 520 KB on-chip SRAM
- GPIO pins: Up to 34 programmable GPIOs

IoT-Enabled ADAS System Using CAN Protocol

- Communication interfaces: UART, SPI, I2C, I2S, CAN (via external controller), PWM, ADC, DAC
- ADC: 12-bit SAR ADC
- DAC: 2×8 -bit DAC channels
- Power modes: Active, Modem-sleep, Light-sleep, Deep-sleep for low power operation
- Development support: Programmable using Arduino IDE, ESP-IDF, and Platform IO.



Fig 2.8 ESP32

3. SOFTWARE REQUIREMENTS

1. STM32CubeIDE.

STM32CubeIDE is a free, all-in-one integrated development environment (IDE) from STMicroelectronics, designed for developing applications on STM32 microcontrollers and microprocessors. It is part of the STM32Cube software ecosystem and is based on the Eclipse/CDT framework. STM32CubeIDE integrates features from STM32CubeMX for configuration and project management, along with a GNU C/C++ compiler toolchain and GDB debugger.

Key features:

- Integration of STM32CubeMX: Allows for the selection of STM32 microcontrollers, pin assignments, clock and peripheral configuration, and initialization code generation.
- Eclipse-based: Supports Eclipse plug-ins and uses the GNU C/C++ toolchain for ARM and GDB debugger.
- Debugging Tools: Offers advanced debugging features, including CPU core, peripheral register, and memory views, live variable watch, system analysis, real-time tracing, and CPU fault analysis.
- Build and Stack Analyzers: Includes build and stack analyzers that provide information on project status and memory requirements.
- Project Import: Supports importing projects from Atollic TrueSTUDIO and AC6 System Workbench for STM32 (SW4STM32).
- Multi-OS Support: Compatible with Windows, Linux, and MacOS.

STM32CubeIDE helps developers with chip selection, project configuration, code generation, editing, compiling, debugging, and burning. It uses a workspace to manage projects, where each workspace is a folder containing project folders and a “. metadata” folder with project information. For dual-core architecture MCUs, such as STM32H7, STM32L5, and STM32MP1 series, the STM32CubeIDE project has a hierarchical structure with "root" and "child" projects for each core.

IoT-Enabled ADAS System Using CAN Protocol

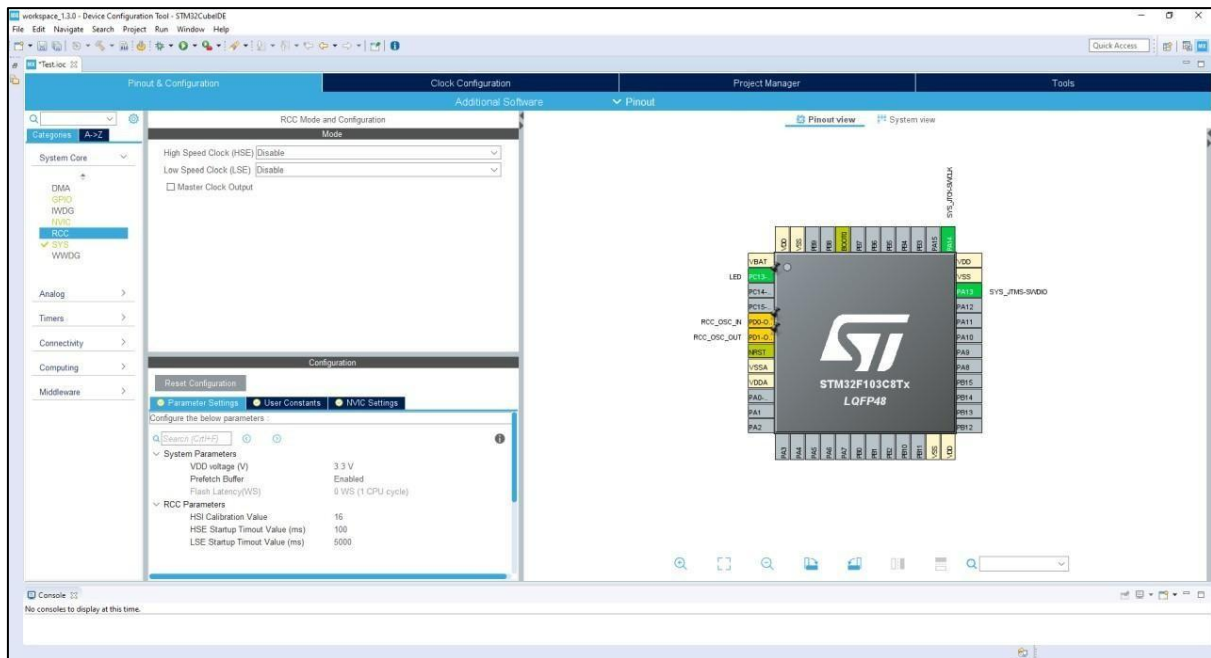


Fig 3.1. STM32CubeIDE UI

2. Arduino IDE

Arduino IDE is a cross-platform, open-source integrated development environment designed for developing applications on microcontroller platforms. It provides a user-friendly interface for writing, compiling, and uploading programs written in C/C++ to supported hardware. In this project, Arduino IDE is used specifically for programming the ESP32 module to enable IoT connectivity and data communication with the ThingSpeak cloud platform.

The Arduino IDE includes a rich set of built-in and third-party libraries that simplify tasks such as Wi-Fi connectivity, HTTP communication, and cloud data transmission. It also offers a serial monitor and serial plotter, which are useful for real-time debugging and data visualization during development. The IDE supports board management through the Boards Manager, allowing easy installation and configuration of ESP32 development packages. Its simplicity, flexibility, and extensive community support make Arduino IDE well-suited for rapid IoT application development and seamless cloud integration in embedded systems projects.



Fig 3.2. Arduino IDE

3. ThingSpeak

ThingSpeak is an IoT cloud platform used for collecting, storing, analyzing, and visualizing data from connected devices over the internet. It allows devices such as microcontrollers and IoT modules to send data to the cloud using standard internet protocols like HTTP. Each project in ThingSpeak uses channels, where sensor data is stored in different fields.

In this project, the ThingSpeak cloud platform is used to receive data from the ESP32 module and store it securely for remote monitoring. ThingSpeak provides built-in tools to visualize data in the form of graphs and charts, making it easy to observe system behavior over time. It also supports data analysis using MATLAB tools, enabling basic processing and interpretation of uploaded data. Due to its simplicity, reliability, and easy integration with Arduino and ESP32, ThingSpeak is widely used in educational and prototype-level IoT applications.

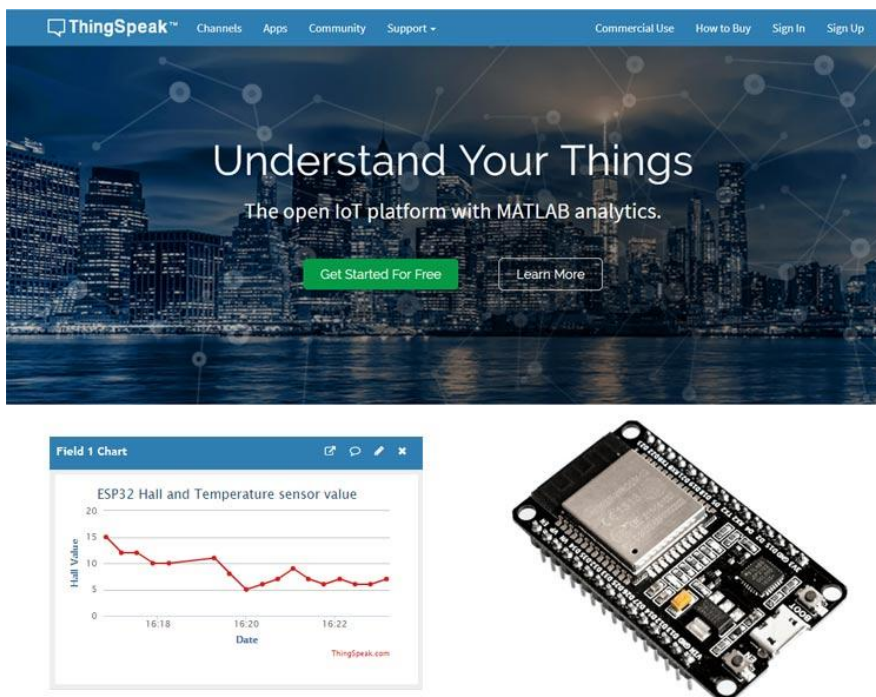


Fig 3.3. ThingSpeak Platform

4. DESIGN AND IMPLEMENTATION

Block Diagram:

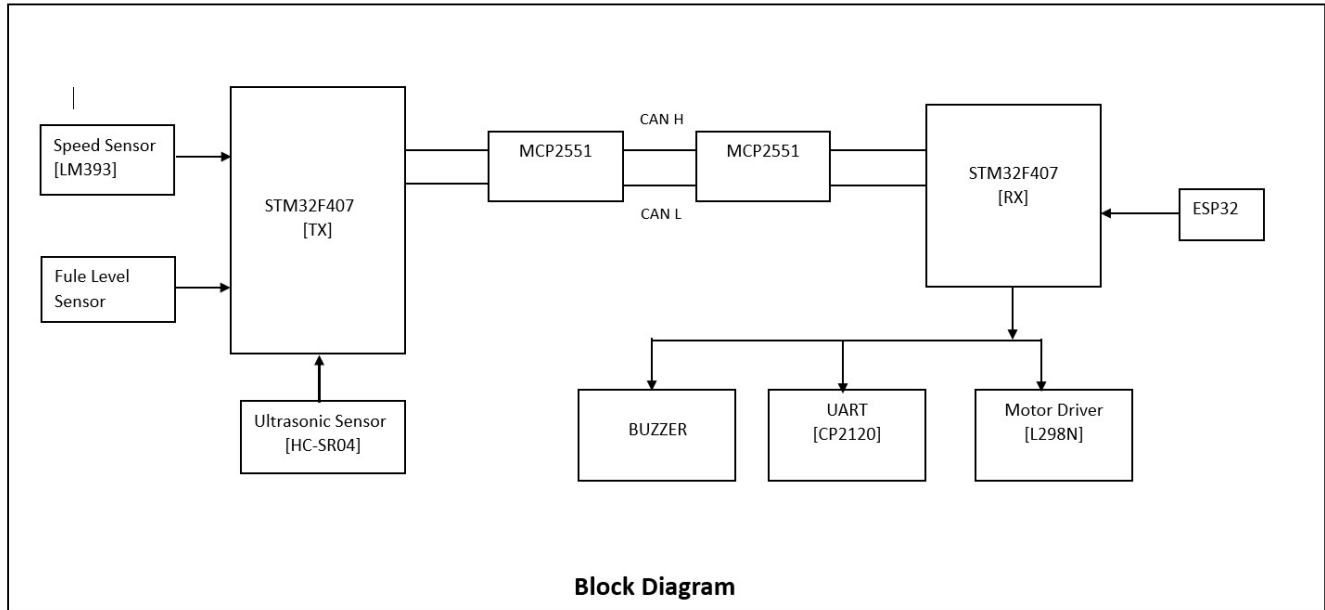


Fig 4.1. Block diagram.

The block diagram represents the architecture of the IoT-Enabled ADAS System using CAN Protocol. The system is divided into two main sections: the Sensor ECU (Transmitter) and the Control ECU (Receiver), which communicate through the CAN bus.

1) Sensor ECU (Transmitter – STM32F407 [TX])

The first STM32F407 microcontroller acts as the Sensor ECU. It collects real-time data from multiple sensors:

Speed Sensor (LM393 based IR sensor): Measures the rotational speed of the motor by detecting pulses and calculating RPM.

Fuel Level Sensor: Measures the liquid level (used as fuel level indication in the system).

Ultrasonic Sensor (HC-SR04): Measures the distance between the vehicle and an obstacle using the time-of-flight principle.

The STM32F407 processes these sensor readings and prepares a CAN data frame containing distance, speed, and fuel level information.

2) CAN Communication (MCP2551 Transceivers)

The processed data is transmitted through the CAN bus using MCP2551 CAN transceivers.

IoT-Enabled ADAS System Using CAN Protocol

One MCP2551 is connected to the transmitter STM32, and another is connected to the receiver STM32.

The CAN bus consists of two differential lines:

CAN H (High)

CAN L (Low)

These lines ensure noise-immune and reliable communication between the two ECUs. The use of differential signaling increases robustness, which is essential for automotive environments.

3) Control ECU (Receiver – STM32F407 [RX])

The second STM32F407 microcontroller acts as the Control ECU. It receives CAN messages from the Sensor ECU and extracts the transmitted parameters such as distance, speed, and fuel level.

Based on the received data, the Control ECU performs safety logic and controls the following components:

- **Motor Driver (L298N):** Controls the DC motor. If the obstacle distance falls below a predefined threshold, the motor is stopped to simulate automatic braking.
- **Buzzer:** Provides an audible alert when a dangerous condition is detected.
- **UART (CP2102/CP2120):** Used for serial communication and real-time debugging. It displays sensor values and system status on a serial monitor.
- **ESP32 Module:** Used for IoT functionality. It sends selected system data to the cloud platform (ThingSpeak) for remote monitoring and data visualization.

5. ADVANTAGES

- **Enhanced safety:** Automatically stops the motor when an obstacle is detected, helping reduce collision risk caused by delayed driver reaction.
- **Reliable communication:** Uses CAN protocol to ensure fast, deterministic, and noise-immune data transfer between sensor ECU and control ECU.
- **Real-time obstacle detection:** Ultrasonic sensor continuously monitors the distance to obstacles and enables timely braking action.
- **Accurate speed monitoring:** IR speed sensor provides real-time RPM information, improving system awareness and control reliability.
- **Reduced vehicle damage:** Early braking action minimizes collision impact, protecting the vehicle and surroundings.
- **Driver monitoring and debugging:** UART output allows real-time observation of distance and speed data for system verification.
- **IoT-enabled monitoring:** ESP32 enables cloud connectivity for remote data monitoring using the ThingSpeak platform.
- **Scalable architecture:** CAN-based modular design allows easy addition of more ECUs or sensors in future enhancements.
- **Cost-effective implementation:** Uses commonly available embedded components, making it suitable for educational and prototype-level automotive applications.

6. FUTURE SCOPE

- Integration of advanced sensors such as radar, LiDAR, or camera modules to improve obstacle detection accuracy and environmental awareness.
- Implementation of intelligent algorithms or machine learning techniques for adaptive braking and decision-making in dynamic traffic conditions.
- Expansion of IoT features for enhanced cloud analytics, predictive maintenance, and real-time alerts.
- Extension of the CAN network to include additional ECUs and dashboard display units for comprehensive vehicle monitoring and control.
- Development of a real-time driver display or mobile application to show alerts, distance, and speed information.
- Integration with vehicle safety systems such as ABS or electronic stability control for more realistic automotive applications.

7. CONCLUSION

The IoT-Enabled ADAS System using CAN Protocol demonstrates an effective approach to improving vehicle safety through automatic obstacle detection and braking. By using CAN-based communication between multiple ECUs, the system ensures reliable and real-time data exchange, which is essential for safety-critical automotive applications. The integration of ultrasonic sensing for obstacle detection and IR-based speed monitoring enables timely decision-making and controlled motor operation to simulate automatic braking.

This project highlights how embedded systems and automotive communication protocols can be combined to implement practical driver assistance features similar to those used in modern vehicles. Although the system is developed at a prototype level, it provides a clear foundation for more advanced ADAS functionalities. With further enhancements such as additional sensors, intelligent algorithms, and deeper IoT integration, the system can be extended toward more sophisticated and real-world automotive safety solutions.

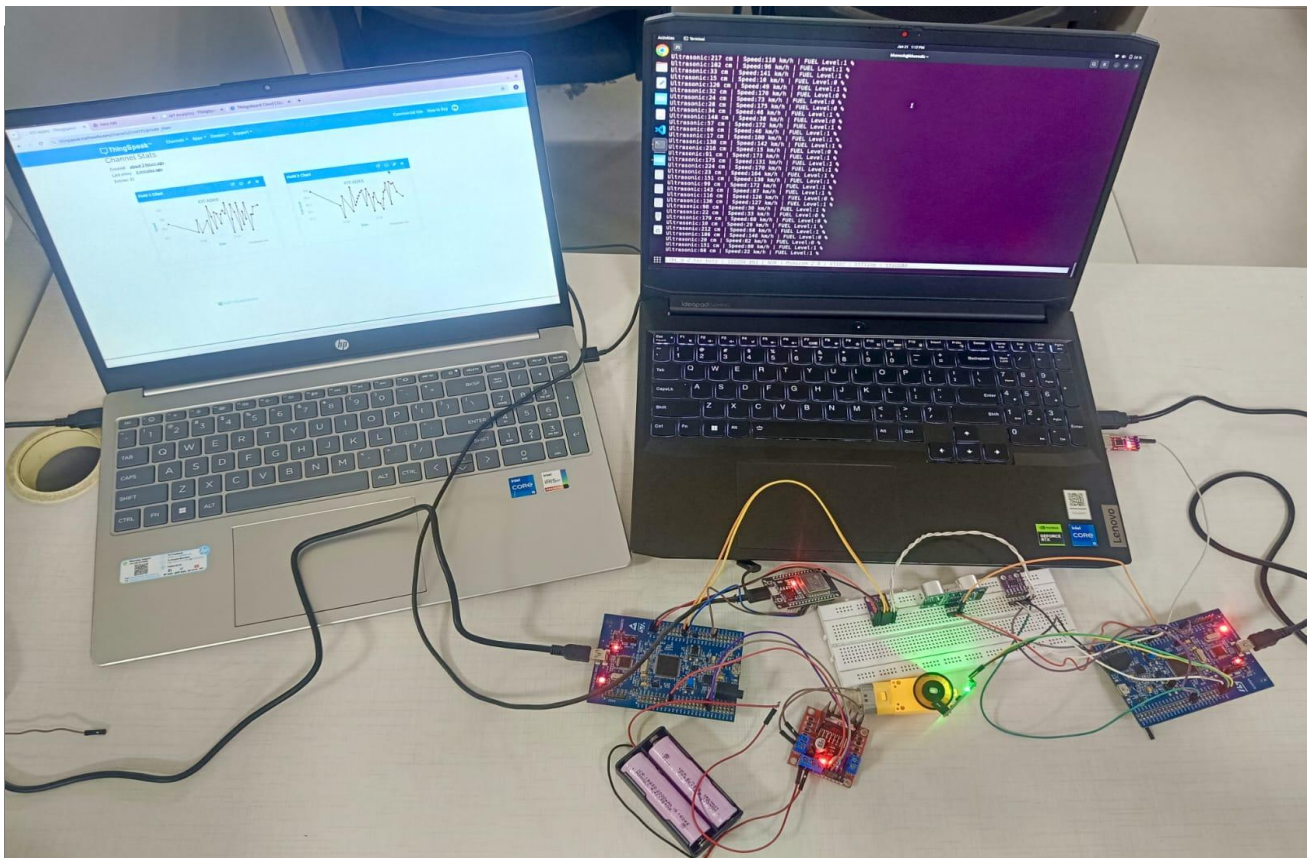
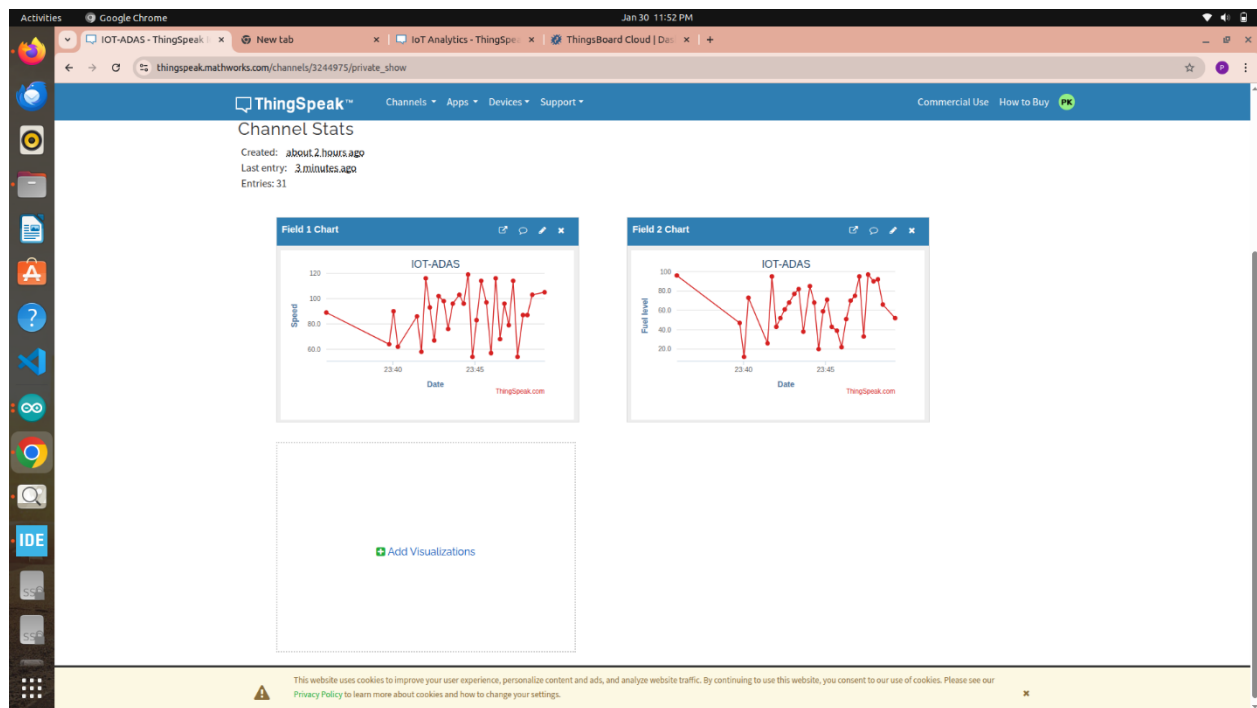
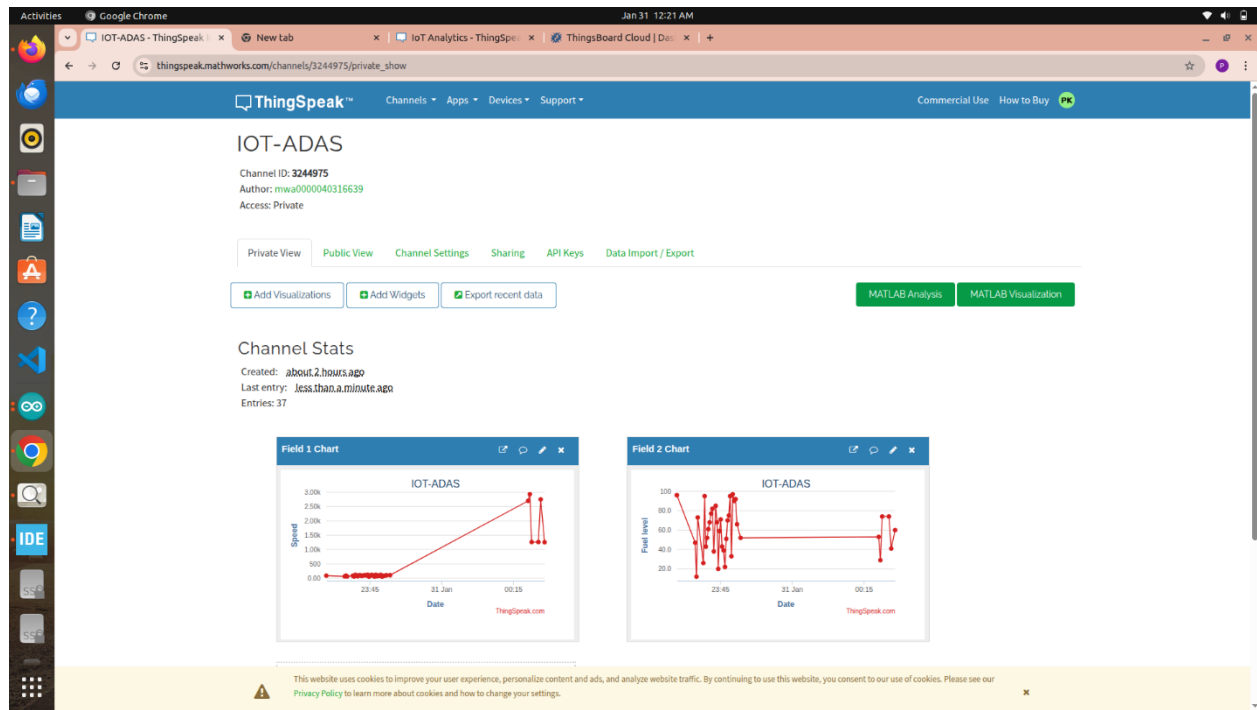


Fig 8.1. IOT Enabled ADAS System.

IoT-Enabled ADAS System Using CAN Protocol



8. REFERENCE

- [1] Robert Bosch GmbH, “CAN Specification Version 2.0,” Bosch, Germany, 1991.
- [2] STMicroelectronics, “STM32F407xx Datasheet,” STMicroelectronics, 2023.
- [3] STMicroelectronics, “STM32F4 Discovery Board User Manual,” STMicroelectronics.
- [4] Microchip Technology Inc., “MCP2551 High-Speed CAN Transceiver Datasheet,” Microchip Technology.
- [5] Espressif Systems, “ESP32 Technical Reference Manual,” Espressif Systems.
- [6] MathWorks, “ThingSpeak IoT Analytics Platform Documentation,” MathWorks.
- [7] Muhammad Ali Mazidi, Rolin D. McKinlay, and Danny Causey, *STM32 ARM Programming for Embedded Systems*, Pearson Education.
- [8] Manoj Prasanth R., Raja S., and Saranya L., “Vehicle Control Using CAN Protocol for Implementing Intelligent Braking System,” *International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering*, Vol. 3, Issue 3, 2014.